

**PANEVROPSKI UNIVERZITET APEIRON, BANJA LUKA
FAKULTET INFORMACIONIH TEHNOLOGIJA**

Redovne studije

Smjer : “Inženjering Informacionih Tehnologija“

Predmet:

Konkurentno računarstvo – paralelno programiranje

**ANALIZA PERFORMANSI SEKVENCIJALNOG I
PARALELNOG IZVRŠAVANJA PROGRAMA U
PYTHONU
(Seminarski rad)**

Predmetni nastavnik

Prof.dr Negovan Stamenković

Student: Nikolina Bekić

Br. Indeksa : 21-22-FIT-IIT1-PRO-240

Banja Luka, jun 2026

SADRŽAJ

UVOD	1
1. OSNOVNI KONCEPTI IZVRŠAVANJA PROGRAMA	2
1.1. Sekvencijalno izvršavanje programa	2
1.2. Razvoj Pythona i njegova primjena	2
1.3. Problem performansi kod velikih skupova podataka	2
1.4. Uloga višejezgrenih procesora u savremenom računarstvu	3
2. KONKURENTNOST I PARALELIZAM U PYTHONU	4
2.1. Razlika između konkurentnosti i paralelizma	4
2.2. Procesi kao nezavisne jedinice izvršavanja	4
2.3. Niti i njihova primjena	4
2.4. Global Interpreter Lock i njegov uticaj na niti	4
2.5. Python biblioteke za paralelno izvršavanje	5
2.5.1. Modul threading	5
2.5.2. Modul multiprocessing	5
2.5.3. Modul concurrent.futures	5
3. PRAKTIČNA KOMPARACIJA SEKVENCIJALNOG I PARALELNOG KODA	6
3.1. Primjer 1: Suma kvadrata brojeva	6
3.2. Primjer 2: Obrada liste ocjena	7
3.3. Primjer 3: Provjera djeljivosti velikog broja elemenata	9
3.4. Primjer 4: Simulacija više sporih operacija	10
3.5. Primjer 5: Analiza više tekstualnih dokumenata	11
3.6. Primjer 6: Računanje prostih brojeva	13
3.7. Primjer 7: Paralelna obrada redova matrice	15
3.8. Primjer 8: Filtriranje podataka iz velike liste	17
3.9. Primjer 9: Generisanje SHA-256 hash vrijednosti	18
3.10. Primjer 10: Simulacija senzorskih očitavanja	19
3.11. Primjer 11: Pretvaranje velikog broja tekstova u velika slova	20
3.12. Primjer 12: Traženje maksimalne vrijednosti u dijelovima liste	21
3.13. Primjer 13: Simulacija Monte Carlo metode	22
3.14. Primjer 14: Sortiranje nezavisnih nizova	24
3.15. Primjer 15: Obrada jednostavne slike kao matrice	26
ZAKLJUČAK	29

UVOD

U savremenom programiranju brzina izvršavanja programa ima veoma važnu ulogu. Programi svakodnevno obrađuju sve veće količine podataka, od jednostavnih tekstualnih fajlova do velikih baza, slika, video zapisa i rezultata naučnih eksperimenata. Zbog toga se od programera ne očekuje samo da napiše tačan program, nego i da razmišlja o načinu na koji se taj program izvršava.

Najjednostavniji oblik izvršavanja programa jeste sekvencijalno izvršavanje. Kod takvog pristupa naredbe se izvršavaju jedna poslije druge. Ovaj pristup je logičan, pregledan i pogodan za većinu osnovnih programskih zadataka. Međutim, problem nastaje kada program treba da obradi veliki broj podataka ili kada se isti tip operacije mora ponoviti mnogo puta. Tada sekvencijalno izvršavanje može postati sporo.

Razvojem modernih procesora, posebno procesora sa više jezgara, pojavila se mogućnost da se više dijelova programa izvršava istovremeno. Takav pristup naziva se paralelno programiranje. Paralelizam omogućava da se posao podijeli na više manjih zadataka i rasporedi na više procesorskih jezgara. Time se u određenim slučajevima može značajno smanjiti ukupno vrijeme izvršavanja.

U ovom radu analizira se razlika između sekvencijalnog i paralelnog programiranja u Python okruženju. Python je izabran zato što je čitljiv, jednostavan za učenje i široko primijenjen u industriji, nauci, obradi podataka i automatizaciji. Zvanična Python dokumentacija opisuje Python kao jezik koji je jednostavan za učenje, ali istovremeno dovoljno moćan za razvoj različitih vrsta aplikacija.

Cilj rada je da se kroz teorijski dio objasne osnovni pojmovi kao što su procesi, niti, konkurentnost i paralelizam, a zatim da se kroz 15 praktičnih primjera uporede sekvencijalni i paralelni pristupi. Za svaki primjer prikazan je opis problema, sekvencijalni kod, paralelni kod i kratka analiza očekivanih rezultata.

1. OSNOVNI KONCEPTI IZVRŠAVANJA PROGRAMA

1.1. Sekvencijalno izvršavanje programa

Sekvencijalno izvršavanje predstavlja osnovni i najjednostavniji način rada programa. Kod ovog pristupa program izvršava naredbe redom kojim su napisane. Kada se jedna naredba završi, prelazi se na sljedeću. Ovakav način izvršavanja je jednostavan za razumijevanje jer odgovara prirodnom načinu razmišljanja.

Na primjer, ako program treba da pročita listu brojeva, izračuna njihov zbir i zatim prikaže rezultat, sve se odvija u jednom toku. Prvo se učitavaju podaci, zatim se obrađuju, a zatim prikazuje rezultat. Programer lako može pratiti šta se dešava u svakom trenutku.

Prednost sekvencijalnog programiranja je njegova jednostavnost. Kod je obično kraći, pregledniji i lakše ga je testirati. Greške se lakše pronalaze jer postoji jedan glavni tok izvršavanja. Zbog toga se sekvencijalni pristup i dalje koristi u velikom broju programa.

Ipak, sekvencijalno izvršavanje ima ograničenja. Ako se isti zadatak mora izvršiti nad velikim brojem elemenata, program može trajati dugo. Takođe, ako računar ima više procesorskih jezgara, sekvencijalni program najčešće neće iskoristiti puni potencijal hardvera.

1.2. Razvoj Pythona i njegova primjena

Python je programski jezik visokog nivoa koji je kreirao Guido van Rossum. Prema zvaničnoj dokumentaciji, Python je nastao početkom devedesetih godina u CWI institutu u Holandiji kao nasljednik programskog jezika ABC.

Danas je Python jedan od najpopularnijih programskih jezika. Koristi se u obrazovanju, web razvoju, automatizaciji, administraciji sistema, analizi podataka, vještačkoj inteligenciji, mašinskom učenju i naučnim istraživanjima. Njegova sintaksa je jednostavna i čitljiva, pa omogućava programerima da se više fokusiraju na rješavanje problema, a manje na tehničke detalje jezika.

Python ima veliki broj standardnih i dodatnih biblioteka. Za temu ovog rada posebno su važne biblioteke koje omogućavaju konkurentno i paralelno izvršavanje, kao što su `threading`, `multiprocessing` i `concurrent.futures`.

1.3. Problem performansi kod velikih skupova podataka

Performanse programa zavise od mnogo faktora. Neki programi su spori zbog lošeg algoritma, neki zbog velike količine podataka, a neki zbog toga što ne koriste dostupne hardverske resurse. Kada program obrađuje mali broj elemenata, razlika između sekvencijalnog i paralelnog izvršavanja često nije značajna. Međutim, kada se broj elemenata poveća, razlika može postati veoma vidljiva.

Tipični problemi kod kojih sekvencijalno izvršavanje može biti sporo su obrada velikih lista, pretraga podataka, obrada više fajlova, obrada slika, simulacije i matematički proračuni. Ako se svaki dio posla može izvršavati nezavisno, paralelizam može biti dobro rješenje.

Važno je naglasiti da paralelizam ne znači automatski ubrzanje. Paralelno izvršavanje uvodi dodatni trošak, kao što je kreiranje niti ili procesa, prenos podataka i sinhronizacija. Zbog toga je neophodno mjeriti vrijeme izvršavanja i uporediti rezultate.

1.4. Uloga višejezgrenih procesora u savremenom računarstvu

Savremeni računari najčešće imaju procesore sa više jezgara. Svako jezgro može izvršavati dio posla, pa se pravilno napisan program može izvršavati brže. Međutim, hardver sam po sebi nije dovoljan. Program mora biti organizovan tako da može koristiti više jezgara.

Ako je program napisan isključivo sekvencijalno, on će uglavnom izvršavati jedan tok instrukcija. To znači da se potencijal višejezgrenog procesora ne koristi u potpunosti. Paralelno programiranje omogućava da se zadatak podijeli, a zatim izvršava istovremeno na više jezgara.

U Pythonu izbor između niti i procesa zavisi od prirode zadatka. Ako zadatak najviše vremena provodi čekajući, na primjer kod fajlova ili mreže, niti mogu biti dobar izbor. Ako je zadatak računski zahtjevan, procesi su često bolji izbor jer mogu zaobići ograničenja Global Interpreter Lock mehanizma.

2. KONKURENTNOST I PARALELIZAM U PYTHONU

2.1. Razlika između konkurentnosti i paralelizma

Konkurentnost i paralelizam su povezani pojmovi, ali ne znače isto. Konkurentnost znači da program može upravljati sa više zadataka u istom vremenskom periodu. Ti zadaci se mogu smjenjivati, ali ne moraju nužno biti izvršeni u istom trenutku.

Paralelizam znači da se više zadataka zaista izvršava istovremeno. Za pravi paralelizam obično je potrebno više procesorskih jezgara. Ako program ima četiri nezavisna zadatka, a računar četiri jezgra, moguće je da se svaki zadatak izvršava na posebnom jezgru. Python podržava oba pristupa. Konkurentnost se često postiže nitima, dok se paralelizam za računski zahtjevne zadatke obično postiže procesima.

2.2. Proces i kao nezavisne jedinice izvršavanja

Proces je nezavisna jedinica izvršavanja programa. Svaki proces ima svoju memoriju i svoje resurse. Zbog toga procesi ne dijele podatke direktno kao niti. Ova izolacija povećava stabilnost, ali komunikaciju između procesa čini složenijom.

U Pythonu se procesi koriste preko modula multiprocessing. Zvanična dokumentacija navodi da multiprocessing podržava kreiranje procesa i omogućava iskorišćavanje više procesora korišćenjem podprocesa umjesto niti.

Procesi su posebno korisni kod CPU-intenzivnih zadataka. Primjeri su računanje prostih brojeva, matematičke simulacije, obrada slika i veći numerički proračuni.

2.3. Niti i njihova primjena

Nit je lakša jedinica izvršavanja unutar procesa. Više niti u jednom procesu dijeli memoriju tog procesa. Zbog toga su niti pogodne kada zadaci treba da komuniciraju ili koriste zajedničke podatke.

Niti su posebno korisne kod I/O zadataka. To su zadaci u kojima program čeka disk, mrežu, korisnički unos ili neki drugi spoljašnji resurs. Dok jedna nit čeka, druga može izvršavati drugi dio posla.

Međutim, rad sa nitima zahtijeva oprez. Ako više niti istovremeno pristupa istim podacima, mogu nastati greške. Zbog toga se često koriste mehanizmi zaključavanja, kao što su Lock, Semaphore i slični alati.

2.4. Global Interpreter Lock i njegov uticaj na niti

Jedna od specifičnosti standardne CPython implementacije jeste Global Interpreter Lock, skraćeno GIL. GIL je mehanizam koji ograničava istovremeno izvršavanje Python bajtkoda u više niti. Zvanična dokumentacija objašnjava da nit mora držati GIL da bi radila sa Python objektima.

Zbog GIL-a, Python niti nisu idealne za računski zahtjevne zadatke. Na primjer, ako dvije niti pokušavaju izvršavati teške matematičke operacije, ne mora se dobiti očekivano ubrzanje. Međutim, kod I/O operacija GIL nije toliko veliki problem, jer se tokom čekanja može omogućiti izvršavanje druge niti.

Zbog toga se u Pythonu često pravi razlika između CPU-bound i I/O-bound zadataka. Za CPU-bound zadatke bolji je multiprocessing, dok su za I/O-bound zadatke često dovoljne niti.

2.5. Python biblioteke za paralelno izvršavanje

Python standardna biblioteka nudi više modula za konkurentno i paralelno izvršavanje. Zvanična dokumentacija navodi da izbor alata zavisi od vrste zadatka, odnosno od toga da li je zadatak vezan za CPU ili za I/O operacije.

2.5.1. Modul threading

Modul threading omogućava kreiranje i upravljanje nitima. Pogodan je za zadatke koji uključuju čekanje, kao što su mrežni zahtjevi, rad sa fajlovima ili simulacija odgođenih operacija.

Prednost ovog modula je jednostavnost i manja potrošnja resursa u odnosu na procese. Nedostatak je ograničenje kod CPU-intenzivnih zadataka zbog GIL-a.

2.5.2. Modul multiprocessing

Modul multiprocessing omogućava pokretanje više procesa. Svaki proces ima sopstveni Python interpreter i memorijski prostor. Zbog toga ovaj modul može bolje iskoristiti višejezgrene procesore kod računski zahtjevnih zadataka.

Nedostatak je veći trošak pokretanja procesa i prenosa podataka. Ako se procesima šalju veoma veliki podaci, paralelna verzija može biti usporena.

2.5.3. Modul concurrent.futures

Modul concurrent.futures pruža jednostavniji interfejs za asinhrono izvršavanje zadataka. On sadrži ThreadPoolExecutor i ProcessPoolExecutor. Dokumentacija navodi da ovaj modul pruža interfejs visokog nivoa za asinhrono izvršavanje pozivljivih objekata.

Ovaj modul je pogodan za praktične primjere jer omogućava jednostavno pokretanje više zadataka bez detaljnog ručnog upravljanja nitima i procesima.

3. PRAKTIČNA KOMPARACIJA SEKVENCIJALNOG I PARALELNOG KODA

3.1. Primjer 1: Suma kvadrata brojeva

Opis problema: Potrebno je izračunati zbir kvadrata brojeva u velikom opsegu.

```
zadatak-01-sekvencionalno.py ×  
1 import time  
2  
3 def sum_of_squares(numbers):  
4     return sum(n * n for n in numbers)  
5  
6 numbers = range(1, 5_000_001)  
7  
8 start = time.perf_counter()  
9 result = sum_of_squares(numbers)  
10 end = time.perf_counter()  
11  
12 print("Rezultat:", result)  
13 print("Sekvencijalno vrijeme:", end - start)  
  
Run zadatak-01-sekvencionalno ×  
C:\Users\Admin\PycharmProjects\PythonProject\.venv  
Rezultat: 41666679166667500000  
Sekvencijalno vrijeme: 0.4004298000363633
```



```
zadatak-01-paralelno.py x
1 import time
2 from concurrent.futures import ProcessPoolExecutor
3 from multiprocessing import cpu_count
4
5 def sum_of_squares(numbers): 1usage
6     return sum(n * n for n in numbers)
7
8 if __name__ == "__main__":
9     numbers = list(range(1, 5_000_001))
10    cores = cpu_count()
11    chunk_size = len(numbers) // cores
12    chunks = [numbers[i:i + chunk_size] for i in range(0, len(numbers), chunk_size)]
13
14    start = time.perf_counter()
15    with ProcessPoolExecutor() as executor:
16        partial_results = list(executor.map(sum_of_squares, *iterables: chunks))
17    result = sum(partial_results)
18    end = time.perf_counter()
19
20    print("Rezultat:", result)
21    print("Paralelno vrijeme:", end - start)
22
Run zadatak-01-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\Py
Rezultat: 41666679166667500000
Paralelno vrijeme: 0.6941026999847963
```

Ovaj zadatak je pogodan za paralelizam jer se zbir kvadrata može izračunati po dijelovima. Svaki proces računa parcijalni rezultat, a zatim se svi rezultati saberu. Kod većeg broja elemenata može se očekivati ubrzanje.

3.2. Primjer 2: Obrada liste ocjena

Opis problema: Potrebno je izračunati koliko ocjena iz liste prelazi određeni prag.

zadatak-02-sekvencionalno.py ×

```
1 import time
2 import random
3
4 grades = [random.randint(a: 5, b: 10) for _ in range(3_000_000)]
5
6 start = time.perf_counter()
7 passed = sum(1 for grade in grades if grade >= 6)
8 end = time.perf_counter()
9
10 print("Broj prolaznih ocjena:", passed)
11 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-02-sekvencionalno ×



C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C
Broj prolaznih ocjena: 2501111
Sekvencijalno vrijeme: 0.11263420002069324

```
zadatak-02-paralelno.py x
1 > import ...
5
6 def count_passed(chunk): 1usage
7     return sum(1 for grade in chunk if grade >= 6)
8
9 ▶ if __name__ == "__main__":
10     grades = [random.randint(a: 5, b: 10) for _ in range(3_000_000)]
11     cores = cpu_count()
12     chunk_size = len(grades) // cores
13     chunks = [grades[i:i + chunk_size] for i in range(0, len(grades), chunk_size)]
14
15     start = time.perf_counter()
16     with ProcessPoolExecutor() as executor:
17         results = list(executor.map(count_passed, *iterables: chunks))
18     passed = sum(results)
19     end = time.perf_counter()
20
21     print("Broj prolaznih ocjena:", passed)
22     print("Paralelno vrijeme:", end - start)
23
Run zadatak-02-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\
Broj prolaznih ocjena: 2500018
Paralelno vrijeme: 0.3885924000060186
```

Zadatak se lako dijeli jer se svaka ocjena posmatra nezavisno. Paralelna verzija može biti brža kod velikih lista, ali kod manjih lista trošak procesa može biti veći od koristi.

3.3. Primjer 3: Provjera djeljivosti velikog broja elemenata

Opis problema: Potrebno je pronaći brojeve koji su djeljivi sa 3, 5 i 7.

```
zadatak-03-sekvencionalno.py x zadatak-03-paralelno.py
1 import time
2
3 numbers = list(range(1, 8_000_001))
4
5 start = time.perf_counter()
6 result = [n for n in numbers if n % 3 == 0 and n % 5 == 0 and n % 7 == 0]
7 end = time.perf_counter()
8
9 print("Broj elemenata:", len(result))
10 print("Sekvencijalno vrijeme:", end - start)
11
Run zadatak-03-sekvencionalno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Use
Broj elemenata: 76190
Sekvencijalno vrijeme: 0.4162074999185279
```

```
zadatak-03-sekvencionalno.py  zadatak-03-paralelno.py x
1  import time
2  from concurrent.futures import ProcessPoolExecutor
3  from multiprocessing import cpu_count
4
5  def filter_numbers(chunk): 1 usage
6      return [n for n in chunk if n % 3 == 0 and n % 5 == 0 and n % 7 == 0]
7
8  if __name__ == "__main__":
9      numbers = list(range(1, 8_000_001))
10     cores = cpu_count()
11     chunk_size = len(numbers) // cores
12     chunks = [numbers[i:i + chunk_size] for i in range(0, len(numbers), chunk_size)]
13
14     start = time.perf_counter()
15     with ProcessPoolExecutor() as executor:
16         parts = list(executor.map(filter_numbers, *iterables: chunks))
17     result = []
18     for part in parts:
19         result.extend(part)
20     end = time.perf_counter()
21
22     print("Broj elemenata:", len(result))
23     print("Paralelno vrijeme:", end - start)

Run  zadatak-03-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\Py
Broj elemenata: 76190
Paralelno vrijeme: 0.9416315000271425
```

Pošto se svaki broj može provjeriti nezavisno, ovaj primjer je prikladan za paralelnu obradu. Ipak, sama operacija djeljivosti nije veoma složena, pa ubrzanje zavisi od veličine liste.

3.4. Primjer 4: Simulacija više sporih operacija

Opis problema: Simulira se nekoliko operacija koje traju po dvije sekunde.

```
zadatak-04-sekvencionalno.py x zadatak-04-paralelno.py x
1 import time
2
3 def slow_operation(i): 1 usage
4     time.sleep(2)
5     return f"Operacija {i} završena"
6
7 start = time.perf_counter()
8 results = [slow_operation(i) for i in range(5)]
9 end = time.perf_counter()
10
11 print(results)
12 print("Sekvencijalno vrijeme:", end - start)

Run zadatak-04-sekvencionalno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-04-sekvencionalno.py
['Operacija 0 završena', 'Operacija 1 završena', 'Operacija 2 završena', 'Operacija 3 završena', 'Operacija 4 završena']
Sekvencijalno vrijeme: 10.00222499994561
```

```
zadatak-04-sekvencionalno.py zadatak-04-paralelno.py x
1 import time
2 from concurrent.futures import ThreadPoolExecutor
3
4 def slow_operation(i): 1 usage
5     time.sleep(2)
6     return f"Operacija {i} završena"
7
8 if __name__ == "__main__":
9     start = time.perf_counter()
10     with ThreadPoolExecutor(max_workers=5) as executor:
11         results = list(executor.map(slow_operation, *iterables: range(5)))
12     end = time.perf_counter()
13
14 print(results)
15 print("Paralelno vrijeme:", end - start)

Run zadatak-04-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-04-paralelno.py
['Operacija 0 završena', 'Operacija 1 završena', 'Operacija 2 završena', 'Operacija 3 završena', 'Operacija 4 završena']
Paralelno vrijeme: 2.0018610999686643
```

Sekvencijalna verzija traje približno deset sekundi. Paralelna verzija traje približno dvije sekunde jer sve operacije čekanja počinju skoro istovremeno. Ovo je dobar primjer upotrebe niti.

3.5. Primjer 5: Analiza više tekstualnih dokumenata

Opis problema: Potrebno je prebrojati broj riječi u više velikih tekstova.

zadatak-05-sekvencionalno.py × zadatak-05-paralelno.py

```
1 import time
2
3 texts = [
4     "Programiranje u Pythonu je korisno. " * 300_000,
5     "Paralelizam ubrzava nezavisne zadatke. " * 300_000,
6     "Sekvencijalni kod se izvrsava redom. " * 300_000
7 ]
8
9 def word_count(text): 1 usage
10     return len(text.split())
11
12 start = time.perf_counter()
13 counts = [word_count(text) for text in texts]
14 end = time.perf_counter()
15
16 print("Ukupno rijeci:", sum(counts))
17 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-05-sekvencionalno ×



↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
↓ Ukupno rijeci: 4200000
— Sekvencijalno vrijeme: 0.31169149989727885

```
zadatak-05-sekvencionalno.py  zadatak-05-paralelno.py x
1  import time
2  from concurrent.futures import ProcessPoolExecutor
3
4  def word_count(text): 1 usage
5      return len(text.split())
6
7  if __name__ == "__main__":
8      texts = [
9          "Programiranje u Pythonu je korisno. " * 300_000,
10         "Paralelizam ubrzava nezavisne zadatke. " * 300_000,
11         "Sekvencijalni kod se izvrsava redom. " * 300_000
12     ]
13
14     start = time.perf_counter()
15     with ProcessPoolExecutor() as executor:
16         counts = list(executor.map(word_count, *iterables: texts))
17     end = time.perf_counter()
18
19     print("Ukupno rijeci:", sum(counts))
20     print("Paralelno vrijeme:", end - start)
21
Run  zadatak-05-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.
Ukupno rijeci: 4200000
Paralelno vrijeme: 0.3954547999892384
```

Svaki tekst se može analizirati odvojeno, pa je zadatak pogodan za paralelno izvršavanje. Kod vrlo velikih tekstova može se dobiti ubrzanje.

3.6. Primjer 6: Računanje prostih brojeva

Opis problema: Potrebno je odrediti koliko brojeva u datom intervalu ima osobinu prostog broja.

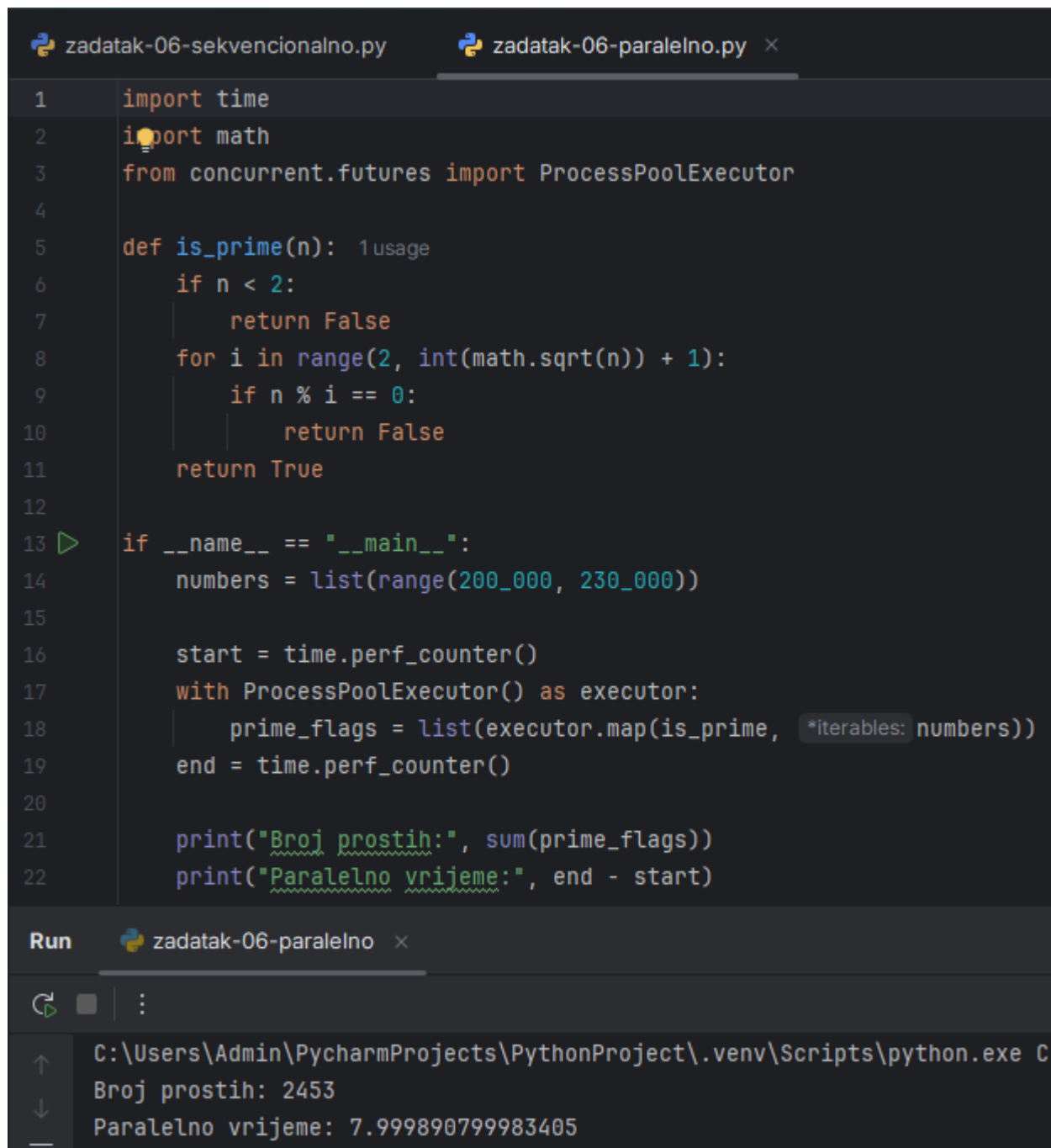
zadatak-06-sekvencionalno.py × zadatak-06-paralelno.py

```
1 import time
2 import math
3
4 def is_prime(n): 1 usage
5     if n < 2:
6         return False
7     for i in range(2, int(math.sqrt(n)) + 1):
8         if n % i == 0:
9             return False
10    return True
11
12 numbers = range(200_000, 230_000)
13
14 start = time.perf_counter()
15 prime_flags = [is_prime(n) for n in numbers]
16 end = time.perf_counter()
17
18 print("Broj prostih:", sum(prime_flags))
19 print("Sekvencijalno vrijeme:", end - start)
--
```

Run zadatak-06-sekvencionalno ×



```
C:\Users\Admin\PycharmProjects\PythonProject\.venv\
Broj prostih: 2453
Sekvencijalno vrijeme: 0.07193869992624968
```

```
zadatak-06-sekvencionalno.py  zadatak-06-paralelno.py x
1  import time
2  import math
3  from concurrent.futures import ProcessPoolExecutor
4
5  def is_prime(n): 1 usage
6      if n < 2:
7          return False
8      for i in range(2, int(math.sqrt(n)) + 1):
9          if n % i == 0:
10             return False
11         return True
12
13  if __name__ == "__main__":
14      numbers = list(range(200_000, 230_000))
15
16      start = time.perf_counter()
17      with ProcessPoolExecutor() as executor:
18          prime_flags = list(executor.map(is_prime, *iterables: numbers))
19      end = time.perf_counter()
20
21      print("Broj prostih:", sum(prime_flags))
22      print("Paralelno vrijeme:", end - start)

Run  zadatak-06-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C
Broj prostih: 2453
Paralelno vrijeme: 7.999890799983405
```

Provjera prostih brojeva je računski zahtjevnija od jednostavne provjere djeljivosti. Zbog toga paralelna verzija ima veću šansu da ostvari ubrzanje.

3.7. Primjer 7: Paralelna obrada redova matrice

Opis problema: Potrebno je sabrati elemente svakog reda velike matrice.

```
zadatak-07-sekvencionalno.py x zadatak-07-paralelno.py
1 import time
2
3 matrix = [[i for i in range(1000)] for _ in range(3000)]
4
5 start = time.perf_counter()
6 row_sums = [sum(row) for row in matrix]
7 end = time.perf_counter()
8
9 print("Broj redova:", len(row_sums))
10 print("Sekvencijalno vrijeme:", end - start)

Run zadatak-07-sekvencionalno x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj redova: 3000
Sekvencijalno vrijeme: 0.021327099995687604
```

```
zadatak-07-sekvencionalno.py zadatak-07-paralelno.py x
1 import time
2 from concurrent.futures import ProcessPoolExecutor
3
4 def sum_row(row): 1 usage
5     return sum(row)
6
7 if __name__ == "__main__":
8     matrix = [[i for i in range(1000)] for _ in range(3000)]
9
10    start = time.perf_counter()
11    with ProcessPoolExecutor() as executor:
12        row_sums = list(executor.map(sum_row, *iterables: matrix))
13    end = time.perf_counter()
14
15    print("Broj redova:", len(row_sums))
16    print("Paralelno vrijeme:", end - start)

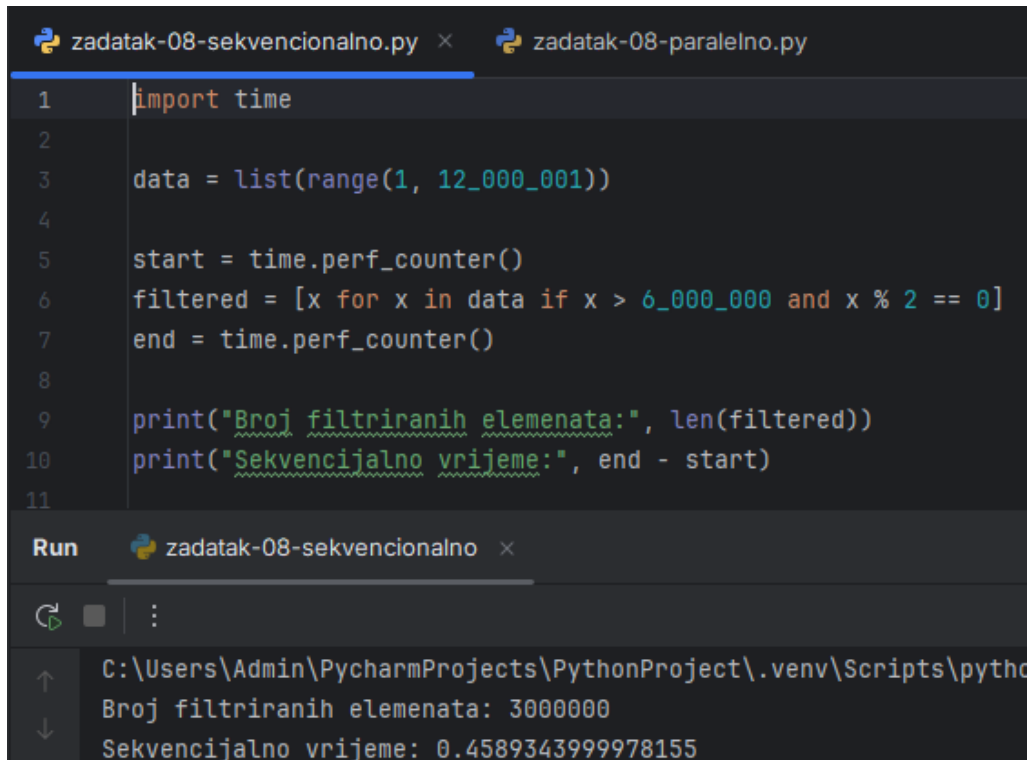
Run zadatak-07-paralelno x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj redova: 3000
Paralelno vrijeme: 0.8341648000059649
```

Svaki red matrice može se obraditi nezavisno. Kod veoma velikih matrica paralelna obrada može biti korisna, dok kod manjih matrica sekvencijalni pristup može biti jednostavniji i brži.

3.8. Primjer 8: Filtriranje podataka iz velike liste

Opis problema: Potrebno je izdvojiti sve brojeve koji su veći od određenog praga i parni.

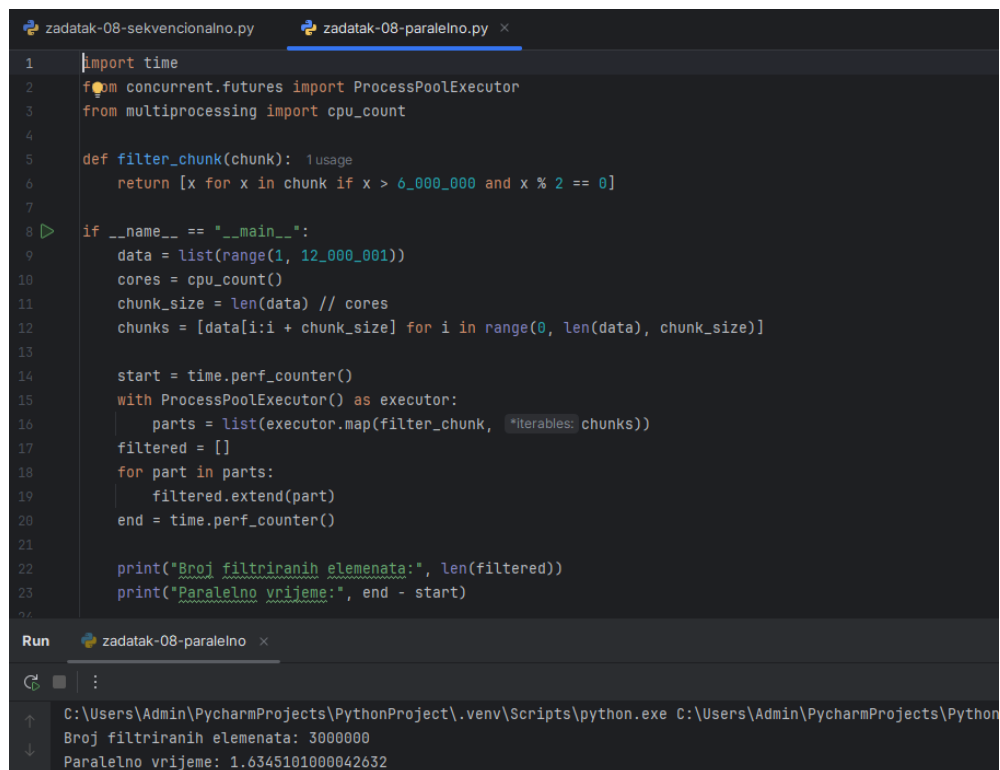


```
zadatak-08-sekvencionalno.py x zadatak-08-paralelno.py
1 import time
2
3 data = list(range(1, 12_000_001))
4
5 start = time.perf_counter()
6 filtered = [x for x in data if x > 6_000_000 and x % 2 == 0]
7 end = time.perf_counter()
8
9 print("Broj filtriranih elemenata:", len(filtered))
10 print("Sekvencijalno vrijeme:", end - start)
11
```

Run zadatak-08-sekvencionalno x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-08-sekvencionalno.py

Broj filtriranih elemenata: 3000000
Sekvencijalno vrijeme: 0.4589343999978155



```
zadatak-08-sekvencionalno.py x zadatak-08-paralelno.py x
1 import time
2 from concurrent.futures import ProcessPoolExecutor
3 from multiprocessing import cpu_count
4
5 def filter_chunk(chunk): 1 usage
6     return [x for x in chunk if x > 6_000_000 and x % 2 == 0]
7
8 if __name__ == "__main__":
9     data = list(range(1, 12_000_001))
10    cores = cpu_count()
11    chunk_size = len(data) // cores
12    chunks = [data[i:i + chunk_size] for i in range(0, len(data), chunk_size)]
13
14    start = time.perf_counter()
15    with ProcessPoolExecutor() as executor:
16        parts = list(executor.map(filter_chunk, *iterables: chunks))
17    filtered = []
18    for part in parts:
19        filtered.extend(part)
20    end = time.perf_counter()
21
22    print("Broj filtriranih elemenata:", len(filtered))
23    print("Paralelno vrijeme:", end - start)
24
```

Run zadatak-08-paralelno x

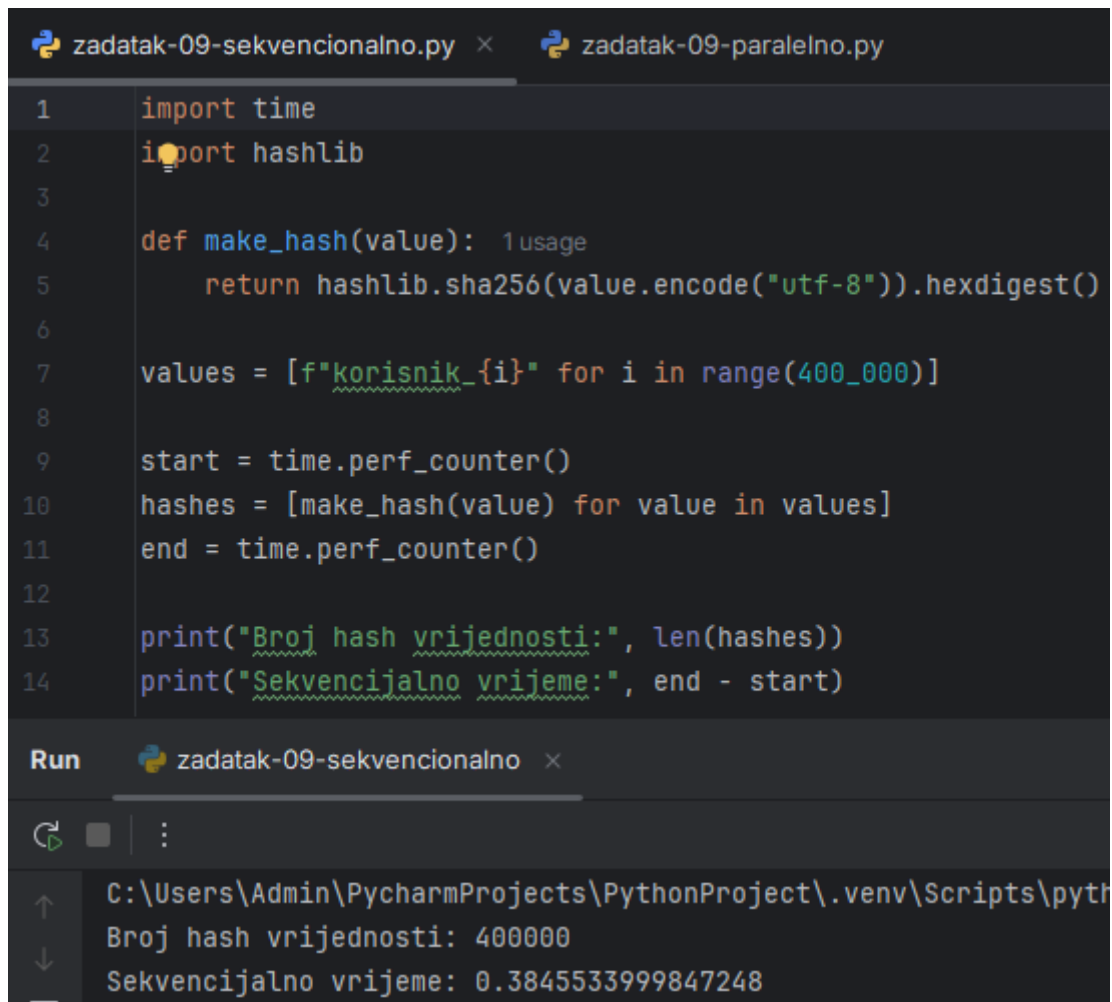
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-08-paralelno.py

Broj filtriranih elemenata: 3000000
Paralelno vrijeme: 1.6345101000042632

Zadatak je jednostavan, ali lista je velika. Paralelna verzija može pomoći ako je količina podataka dovoljno velika. Ako nije, dodatni trošak paralelizacije može smanjiti efekat.

3.9. Primjer 9: Generisanje SHA-256 hash vrijednosti

Opis problema: Potrebno je izračunati hash vrijednosti za veliki broj stringova.



```
zadatak-09-sekvencionalno.py × zadatak-09-paralelno.py
1 import time
2 import hashlib
3
4 def make_hash(value): 1 usage
5     return hashlib.sha256(value.encode("utf-8")).hexdigest()
6
7 values = [f"korisnik_{i}" for i in range(400_000)]
8
9 start = time.perf_counter()
10 hashes = [make_hash(value) for value in values]
11 end = time.perf_counter()
12
13 print("Broj hash vrijednosti:", len(hashes))
14 print("Sekvencijalno vrijeme:", end - start)

Run zadatak-09-sekvencionalno ×
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj hash vrijednosti: 400000
Sekvencijalno vrijeme: 0.3845533999847248
```

```
zadatak-09-sekvencionalno.py x zadatak-09-paralelno.py x
1 import time
2 import hashlib
3 from concurrent.futures import ProcessPoolExecutor
4
5 def make_hash(value): 1 usage
6     return hashlib.sha256(value.encode("utf-8")).hexdigest()
7
8 if __name__ == "__main__":
9     values = [f"korisnik_{i}" for i in range(400_000)]
10
11     start = time.perf_counter()
12
13     with ProcessPoolExecutor() as executor:
14         hashes = list(executor.map(make_hash, *iterables: values, chunksize=1000))
15
16     end = time.perf_counter()
17
18     print("Broj hash vrijednosti:", len(hashes))
19     print("Paralelno vrijeme:", end - start)

Run zadatak-09-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\
Broj hash vrijednosti: 400000
Paralelno vrijeme: 0.504724099992937
```

Hashiranje velikog broja elemenata može biti pogodno za paralelizam. Ipak, kod kratkih stringova trošak slanja podataka procesima može biti značajan.

3.10. Primjer 10: Simulacija senzorskih očitavanja

Opis problema: Simulira se čitanje podataka sa više senzora, gdje svaki senzor kasni jednu sekundu.

```
zadatak-10-sekvencionalno.py x zadatak-10-paralelno.py
1 import time
2 import random
3
4 def read_sensor(sensor_id): 1 usage
5     time.sleep(1)
6     return sensor_id, random.uniform(a: 20.0, b: 30.0)
7
8 start = time.perf_counter()
9 readings = [read_sensor(i) for i in range(6)]
10 end = time.perf_counter()
11
12 print(readings)
13 print("Sekvencijalno vrijeme:", end - start)

Run zadatak-10-sekvencionalno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-10-sekvencionalno.py
[(0, 28.984633888641213), (1, 23.775812427471774), (2, 26.464914426321293), (3, 29.97270445469168), (4, 23.537917410379674), (5, 22.683509315712094)]
Sekvencijalno vrijeme: 6.001502699917182
```

```
zadatak-10-sekvencionalno.py x zadatak-10-paralelno.py x
1 import time
2 import random
3 from concurrent.futures import ThreadPoolExecutor
4
5 def read_sensor(sensor_id): 1 usage
6     time.sleep(1)
7     return sensor_id, random.uniform(a: 20.0, b: 30.0)
8
9 if __name__ == "__main__":
10     start = time.perf_counter()
11     with ThreadPoolExecutor(max_workers=6) as executor:
12         readings = list(executor.map(read_sensor, *iterables: range(6)))
13     end = time.perf_counter()
14
15     print(readings)
16     print("Paralelno vrijeme:", end - start)
17
Run zadatak-10-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-10-paralelno.py
[(0, 23.917905862451228), (1, 25.97885239785358), (2, 29.62740518169517), (3, 21.3118726050321), (4, 27.285601468180207), (5, 20.62352919238168)]
Paralelno vrijeme: 1.001906399964355
```

Ovaj primjer pokazuje tipičan I/O scenario. Pošto se čeka odgovor senzora, niti mogu značajno smanjiti ukupno vrijeme izvršavanja.

3.11. Primjer 11: Pretvaranje velikog broja tekstova u velika slova

Opis problema: Potrebno je transformisati veliki broj tekstualnih elemenata.

```
zadatak-11-sekvencionalno.py x zadatak-11-paralelno.py
1 import time
2
3 texts = [f"tekstualni podatak broj {i}" for i in range(1_000_000)]
4
5 start = time.perf_counter()
6 upper_texts = [text.upper() for text in texts]
7 end = time.perf_counter()
8
9 print("Broj elemenata:", len(upper_texts))
10 print("Sekvencijalno vrijeme:", end - start)

Run zadatak-11-sekvencionalno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj elemenata: 1000000
Sekvencijalno vrijeme: 0.08230060001369566
```

```
zadatak-11-sekvencionalno.py x zadatak-11-paralelno.py x
1 import time
2 from concurrent.futures import ProcessPoolExecutor
3
4 def to_upper(text):
5     return text.upper()
6
7 if __name__ == "__main__":
8     texts = [f"tekstualni podatak broj {i}" for i in range(1_000_000)]
9
10    start = time.perf_counter()
11
12    with ProcessPoolExecutor() as executor:
13        upper_texts = list(executor.map(to_upper, *iterables: texts, chunksize=1000))
14
15    end = time.perf_counter()
16
17    print("Broj elemenata:", len(upper_texts))
18    print("Paralelno vrijeme:", end - start)
```

Run zadatak-11-paralelno x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmP
Broj elemenata: 1000000
Paralelno vrijeme: 0.8727903999388218

Iako se posao može podijeliti, operacija pretvaranja u velika slova je relativno jednostavna. Zbog toga paralelna verzija ne mora uvijek biti brža.

3.12. Primjer 12: Traženje maksimalne vrijednosti u dijelovima liste

Opis problema: Potrebno je pronaći najveći broj u velikoj listi.

```
zadatak-12-sekvencionalno.py x zadatak-12-paralelno.py x
1 import time
2 import random
3
4 numbers = [random.randint(a=1, b=100_000_000) for _ in range(5_000_000)]
5
6 start = time.perf_counter()
7 maximum = max(numbers)
8 end = time.perf_counter()
9
10 print("Maksimum:", maximum)
11 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-12-sekvencionalno x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-12-sekvencionalno.py
Maksimum: 99999936
Sekvencijalno vrijeme: 0.0578094000229612

```
zadatak-12-sekvencionalno.py  zadatak-12-paralelno.py x
1  import time
2  import random
3  from concurrent.futures import ProcessPoolExecutor
4  from multiprocessing import cpu_count
5
6  def chunk_max(chunk):
7      return max(chunk)
8
9  if __name__ == "__main__":
10     numbers = [random.randint(a: 1, b: 100_000_000) for _ in range(5_000_000)]
11     cores = cpu_count()
12     chunk_size = len(numbers) // cores
13     chunks = [numbers[i:i + chunk_size] for i in range(0, len(numbers), chunk_size)]
14
15     start = time.perf_counter()
16     with ProcessPoolExecutor() as executor:
17         partial_maximums = list(executor.map(chunk_max, *iterables: chunks))
18     maximum = max(partial_maximums)
19     end = time.perf_counter()
20
21     print("Maksimum:", maximum)
22     print("Paralelno vrijeme:", end - start)
23
Run  zadatak-12-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\Py
Maksimum: 99999996
Paralelno vrijeme: 0.5904614999890327
```

Lista se može podijeliti na dijelove, a maksimum svakog dijela pronaći odvojeno. Na kraju se traži maksimum među parcijalnim maksimumima. Ovo je logičan primjer dijeljenja problema.

3.13. Primjer 13: Simulacija Monte Carlo metode

Opis problema: Monte Carlo metodom se procjenjuje vrijednost broja pi pomoću slučajno generisanih tačaka.

zadatak-13-sekvencionalno.py × zadatak-13-paralelno.py

```
1 import time
2 import random
3
4 def simulate(points): 1 usage
5     inside = 0
6     for _ in range(points):
7         x = random.random()
8         y = random.random()
9         if x * x + y * y <= 1:
10             inside += 1
11     return inside
12
13 points = 2_500_000
14
15 start = time.perf_counter()
16 inside = simulate(points)
17 pi_estimate = 4 * inside / points
18 end = time.perf_counter()
19
20 print("Procjena broja pi:", pi_estimate)
21 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-13-sekvencionalno ×



C:\Users\Admin\PycharmProjects\PythonProject\.venv\Sc
Procjena broja pi: 3.1431632
Sekvencijalno vrijeme: 0.4033582000993192

```
zadatak-13-sekvencionalno.py x zadatak-13-paralelno.py x
1 import time
2 import random
3 from concurrent.futures import ProcessPoolExecutor
4 from multiprocessing import cpu_count
5
6 def simulate(points): 1 usage
7     inside = 0
8     for _ in range(points):
9         x = random.random()
10        y = random.random()
11        if x * x + y * y <= 1:
12            inside += 1
13    return inside
14
15 if __name__ == "__main__":
16     total_points = 2_500_000
17     cores = cpu_count()
18     points_per_worker = total_points // cores
19
20     start = time.perf_counter()
21     with ProcessPoolExecutor() as executor:
22         results = list(executor.map(simulate, *iterables: [points_per_worker] * cores))
23     inside = sum(results)
24     pi_estimate = 4 * inside / total_points
25     end = time.perf_counter()
26
27     print("Procjena broja pi:", pi_estimate)
28     print("Paralelno vrijeme:", end - start)

Run zadatak-13-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\Py
Procjena broja pi: 3.142632
Paralelno vrijeme: 0.35771759995259345
```

Monte Carlo simulacija je veoma pogodna za paralelizam jer se svaka tačka generiše nezavisno. Zbog toga paralelna verzija može ostvariti dobro ubrzanje.

3.14. Primjer 14: Sortiranje nezavisnih nizova

Opis problema: Potrebno je sortirati više nezavisnih nizova.

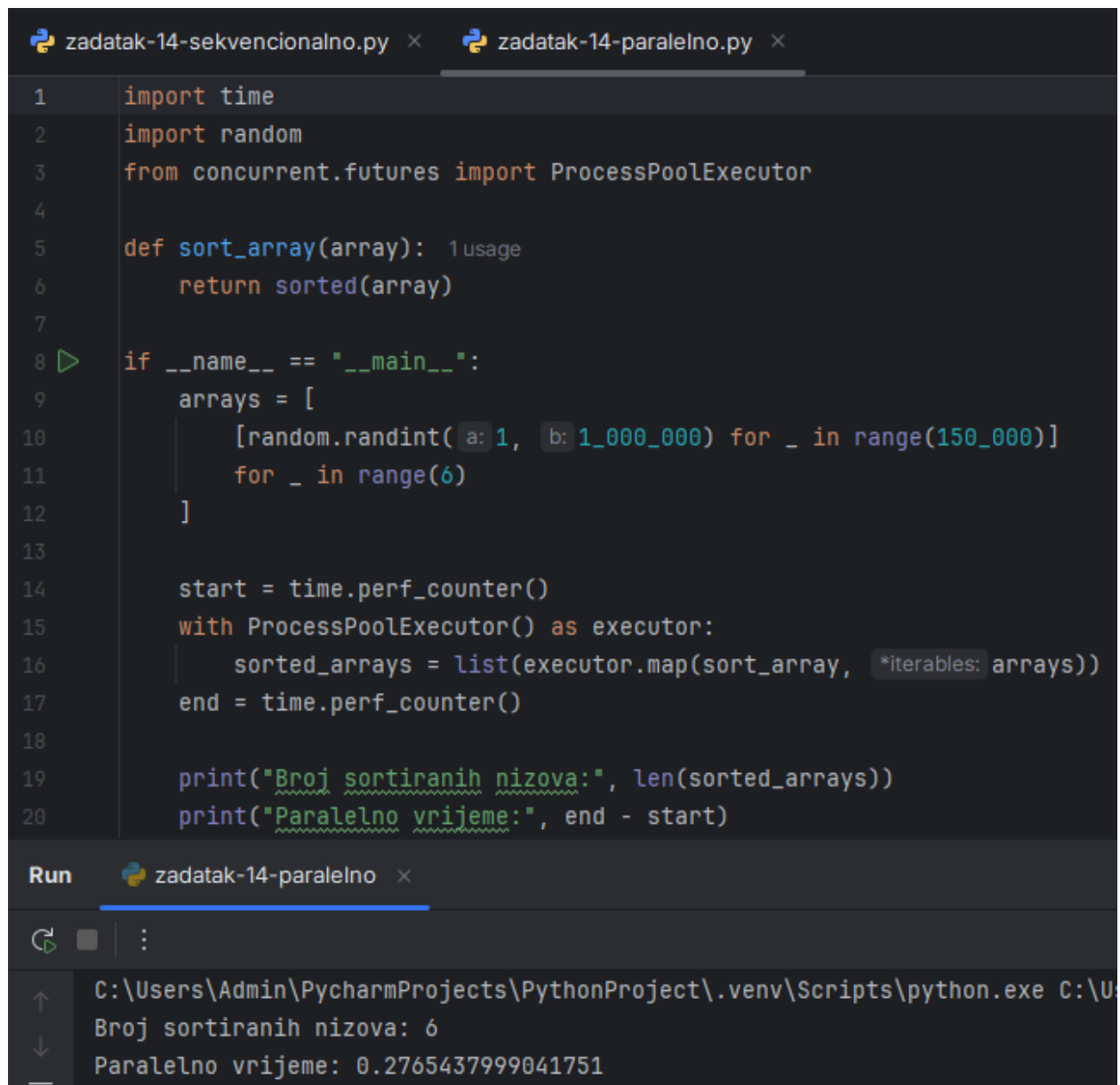
zadatak-14-sekvencionalno.py x zadatak-14-paralelno.py x

```
1 import time
2 import random
3
4 arrays = [
5     [random.randint(a: 1, b: 1_000_000) for _ in range(150_000)]
6     for _ in range(6)
7 ]
8
9 start = time.perf_counter()
10 sorted_arrays = [sorted(array) for array in arrays]
11 end = time.perf_counter()
12
13 print("Broj sortiranih nizova:", len(sorted_arrays))
14 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-14-sekvencionalno x

⏮ ⏪ ⏩ ⏭ ⋮

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
↓ Broj sortiranih nizova: 6
— Sekvencijalno vrijeme: 0.13900970004033297



```
zadatak-14-sekvencionalno.py x zadatak-14-paralelno.py x
1 import time
2 import random
3 from concurrent.futures import ProcessPoolExecutor
4
5 def sort_array(array): 1 usage
6     return sorted(array)
7
8 if __name__ == "__main__":
9     arrays = [
10         [random.randint(a: 1, b: 1_000_000) for _ in range(150_000)]
11         for _ in range(6)
12     ]
13
14     start = time.perf_counter()
15     with ProcessPoolExecutor() as executor:
16         sorted_arrays = list(executor.map(sort_array, *iterables: arrays))
17     end = time.perf_counter()
18
19     print("Broj sortiranih nizova:", len(sorted_arrays))
20     print("Paralelno vrijeme:", end - start)
```

Run zadatak-14-paralelno x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\U
Broj sortiranih nizova: 6
Paralelno vrijeme: 0.2765437999041751

Svaki niz se sortira nezavisno. Zbog toga se sortiranje može rasporediti na više procesa. Ubrzanje zavisi od veličine nizova i broja procesorskih jezgara.

3.15. Primjer 15: Obrada jednostavne slike kao matrice

Opis problema: Slika je predstavljena kao matrica piksela. Potrebno je povećati svaku vrijednost piksela, ali tako da ne pređe 255.

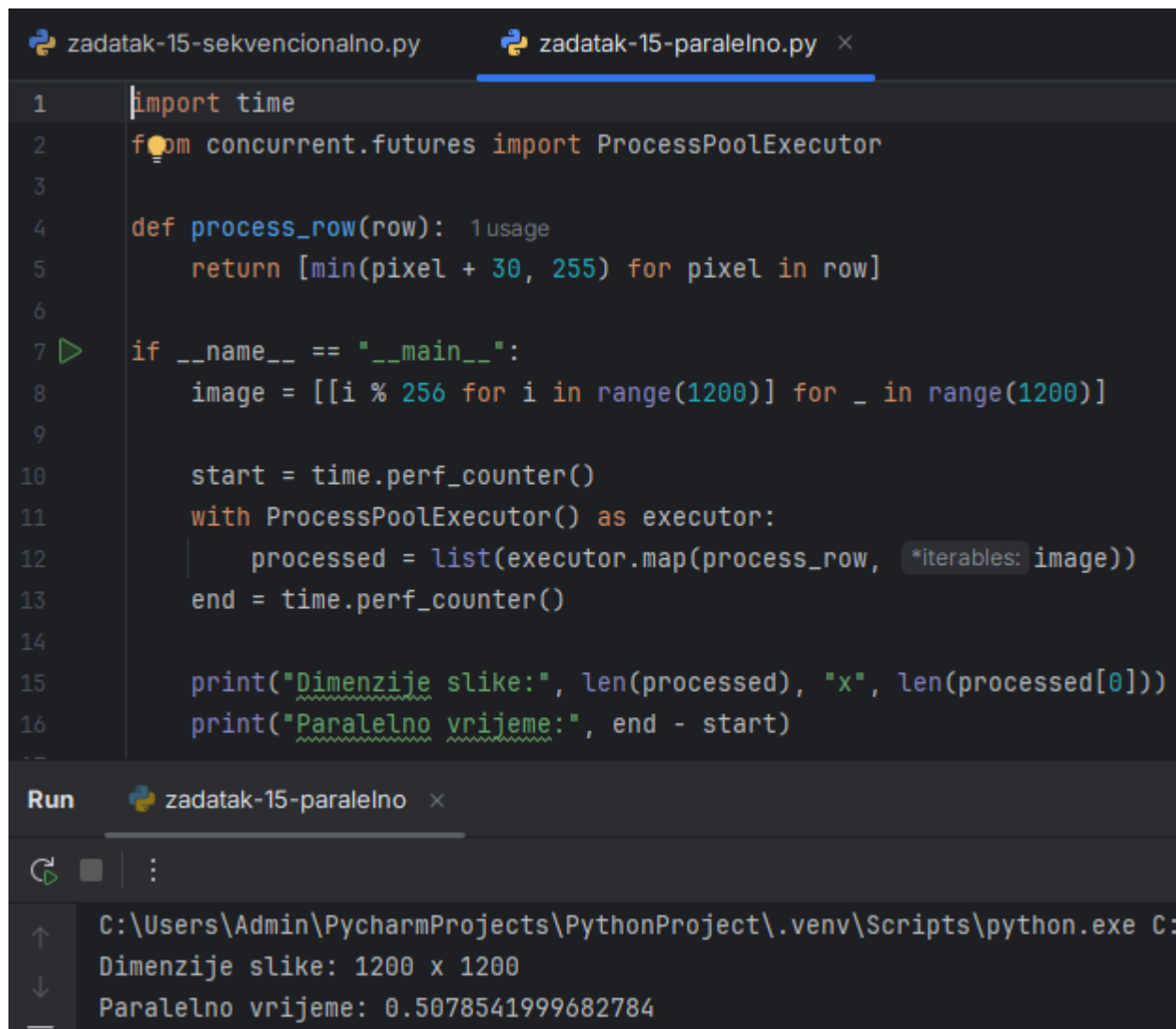
zadatak-15-sekvencionalno.py × zadatak-15-paralelno.py

```
1 import time
2
3 image = [[i % 256 for i in range(1200)] for _ in range(1200)]
4
5 start = time.perf_counter()
6 processed = [
7     [min(pixel + 30, 255) for pixel in row]
8     for row in image
9 ]
10 end = time.perf_counter()
11
12 print("Dimenzije slike:", len(processed), "x", len(processed[0]))
13 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-15-sekvencionalno ×



C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\
Dimenzije slike: 1200 x 1200
Sekvencijalno vrijeme: 0.1150035000173375



```
1 import time
2 from concurrent.futures import ProcessPoolExecutor
3
4 def process_row(row):
5     return [min(pixel + 30, 255) for pixel in row]
6
7 if __name__ == "__main__":
8     image = [[i % 256 for i in range(1200)] for _ in range(1200)]
9
10    start = time.perf_counter()
11    with ProcessPoolExecutor() as executor:
12        processed = list(executor.map(process_row, image))
13    end = time.perf_counter()
14
15    print("Dimenzije slike:", len(processed), "x", len(processed[0]))
16    print("Paralelno vrijeme:", end - start)
```

Run zadatak-15-paralelno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:
Dimenzije slike: 1200 x 1200
Paralelno vrijeme: 0.5078541999682784

Obrada slike je često pogodna za paralelizam jer se redovi ili dijelovi slike mogu obrađivati nezavisno. Kod većih slika paralelna verzija može biti korisna, dok kod manjih slika dodatni trošak može biti izražen.

ZAKLJUČAK

Sekvencijalno programiranje je osnovni i najjednostavniji oblik izvršavanja programa. Njegova glavna prednost je jednostavnost, preglednost i lakše testiranje. Međutim, kod velikih skupova podataka i računski zahtjevnih zadataka sekvencijalni pristup može biti spor.

Paralelno programiranje omogućava podjelu posla na više nezavisnih dijelova. Na taj način program može bolje iskoristiti višejezgrene procesore i smanjiti vrijeme izvršavanja. U Pythonu se za ove potrebe koriste moduli `threading`, `multiprocessing` i `concurrent.futures`.

Kroz praktične primjere prikazano je da paralelizam nije uvijek bolji, ali može biti veoma koristan kada se koristi u odgovarajućim situacijama. Kod I/O zadataka, kao što su čekanje operacija ili simulacija senzora, niti daju dobre rezultate. Kod CPU-intenzivnih zadataka, kao što su prosti brojevi, Monte Carlo simulacija i obrada matrica, bolji izbor su procesi.

Najvažniji zaključak rada je da izbor između sekvencijalnog i paralelnog programiranja mora biti zasnovan na prirodi problema. Paralelno programiranje donosi veću složenost, ali kada se pravilno primijeni, može značajno poboljšati performanse programa.